

A Robust and Generalized Gauss-Seidel Solver for Physically-Correct Simultaneous Collisions

HUANBO ZHOU, Software Engineering Institute, China

ZHENYU XU, Software Engineering Institute, China

XIJUN LIU, Software Engineering Institute, China

XINYU ZHANG*, Software Engineering Institute, China

Simulating multi-object collisions in real-time environments remains a significant challenge, particularly when handling simultaneous collisions in a physically accurate manner. Traditional Gauss-Seidel solvers, widely employed in physics engines, often fail to preserve the symmetry and consistency of multi-object interactions that are often observed in physics. In this paper, we present a generalized and robust Gauss-Seidel solver to overcome the difficulties in simultaneous collisions. Using spatial and temporal collision states to classify and resolve constraints, our algorithm ensures correct collision propagation and symmetry, addressing the limitations commonly found in existing solvers. Moreover, our algorithm can mitigate jitters caused by floating-point errors, ensuring smooth and stable collision responses. Our approach demonstrates fast convergence and improved accuracy in scenarios involving complex multi-object collisions.

CCS Concepts: • **Computing methodologies** → **Physical simulation**.

Additional Key Words and Phrases: Physically-based simulation, Gauss-Seidel solver, Simultaneous collisions

ACM Reference Format:

Huanbo Zhou, Zhenyu Xu, Xijun Liu, and Xinyu Zhang. 2025. A Robust and Generalized Gauss-Seidel Solver for Physically-Correct Simultaneous Collisions. *Proc. ACM Comput. Graph. Interact. Tech.* 8, 1, Article 5 (May 2025), 17 pages. <https://doi.org/10.1145/3728291>

1 INTRODUCTION

Collision and response are critical components in physics simulation, since they underpin the realism and interactivity that define immersive experiences. According to physics laws, collision results between two rigid objects can be uniquely determined by the conservation of momentum and kinetic energy. When collisions involve more than two objects (see Fig. 1), many challenges arise, from simultaneous collision resolution to the order and propagation of collisions [Guendelman et al. 2003; Hahn 1988; Moore and Wilhelms 1988]. To generate physically plausible and correct motions, additional constraints have to be incorporated to obtain sufficient dynamic equations [Glocker 2004; Stewart 2000] and, therefore, collision phenomena can vary significantly depending on the given constraints.

In rigid body simulation, a motion is often divided into discrete time intervals and collisions are found within these intervals. Computing the exact time of collisions (e.g. using continuous collision detection) allows the simulation to proceed in a precise sequence [Bridson et al. 2002; Redon et al. 2005; Zhang et al. 2007]. However, computational performance can decrease significantly as the number of objects increases, making it challenging to scale for scenarios with many interacting

Authors' addresses: Huanbo Zhou, Software Engineering Institute, Shanghai, Shanghai, China, 51255902156@stu.ecnu.edu.cn; Zhenyu Xu, Software Engineering Institute, Shanghai, Shanghai, China, 52265902023@stu.ecnu.edu.cn; Xijun Liu, Software Engineering Institute, Shanghai, Shanghai, China, 1533018670@qq.com; Xinyu Zhang, Software Engineering Institute, Shanghai, Shanghai, China, xyzhang@sei.ecnu.edu.cn.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, <https://doi.org/10.1145/3728291>.

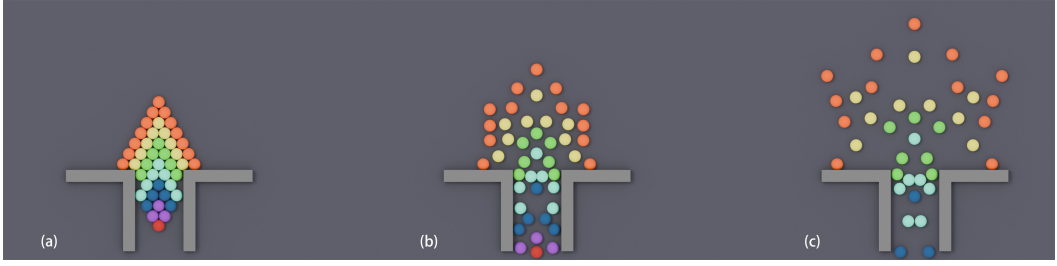


Fig. 1. Our algorithm guarantees the symmetry property during impact propagation in the case of simultaneous and symmetric multiple collision. A set of balls is symmetrically positioned between two stationary obstacles. A red ball is moving upward along the axis of symmetry, striking other balls. (a) The red ball is just striking the two pink balls at the bottom. (b) The balls in the same color maintain their symmetry during impact propagation. The balls near the red ball move backwards, the balls in the middle remain nearly stationary, and the balls in orange, yellow, and green at the top move forward symmetrically. (c) Symmetrical impacts are correctly propagating during simulation.

objects. Therefore, collisions are only considered at discrete time instants (e.g. using discrete collision detection). However, *simultaneous collisions* occur much more frequently than in continuous solutions [Nguyen and Brogliato 2014; Vouga et al. 2017]. Common time discretizations are usually uniform, but there are finer-grained divisions of the step size, such as substepping [Macklin et al. 2019]. In addition, discretization can also be nonuniform, such as asynchronous solvers [Harmon et al. 2009; Zhao et al. 2016].

There are two prevalent collision response methods: *simultaneous model* and *propagation model*. The former treats all collision constraints within a single time step as occurring simultaneously, thus requiring the simulation to satisfy all constraints simultaneously. In contrast, the latter considers the collisions to occur sequentially while also resolving all violated constraints within a single time step.

The simultaneous model treats contacts within a single moment as ineliminable and consequently, objects with zero relative velocity retain relatively static after collision resolution. This causes a *stickiness* phenomenon between objects. As a result, the simultaneous model fails to simulate impulse propagation (e.g., Newton's Cradle [Smith et al. 2012]). Conversely, the propagation model performs well for sequential collisions and can generate the phenomenon of collision propagation between objects. However, it struggles with simultaneous and symmetric collisions (e.g., Bernoulli's problem [Smith et al. 2012] or billiards) and often generates asymmetric collision flaws in scenarios exhibiting symmetry.

Due to the inherent flaws in individual models, hybrid methods have attracted attention [Chen et al. 2023; Smith et al. 2012] by combining two models and leveraging the strengths of each. Hybrid methods simultaneously fulfill the criteria of symmetry preservation and propagation [Smith et al. 2012], therefore leading to more accurate results. For simultaneous collisions, the major challenge is to handle collision cycles. For example, in Newton's cradle, a collision triggers a chain of reactions, leading to many new collisions. A straightforward strategy is to employ iterative solvers that address all collisions within a "frozen moment". Prominent examples of such solvers include the Gauss-Seidel and Jacobi methods [Erleben 2007]. The Gauss-Seidel solver has been widely used in physics engines [Cottle et al. 2009; Erleben 2007; Vouga et al. 2017] due to its robustness and fast convergence. However, the Gauss-Seidel may fail to guarantee the property of symmetry for symmetric and simultaneous collisions. This severely limits its usability for multi-object systems exhibiting many simultaneous collisions.

In addition, the solver also needs to consider the convergence problem. For an iterative solver, if it fails to converge within a given number of iterations, it may introduce unrealistic behavior to the scene. Therefore, whether it can ensure that dynamic properties of the solution are not destroyed when computing power is insufficient is also a key point to measure the robustness of the solver.

Main Results: In this paper, we present a robust and generalized Gauss-Seidel solver to overcome difficulties in simultaneous collisions. By leveraging the inherent mechanism of Projected Gauss-Seidel solver, we carefully and gracefully incorporate additional clues about the propagation and simultaneity of contact constraints. We leverage spatial and temporal collision states (or collision trends) to classify and solve constraints. Not only can we guarantee correct symmetry and collision propagation, but we also achieve fast convergence and reduce the jitters caused by floating-point errors. We also provides corresponding measures for insufficient iterations and parameter control of propagation phenomena. This generalized Gauss-Seidel solver with the integration of propagation and simultaneity represents a significant advancement for the handling of complex contact interactions.

The rest of the paper is organized as follows. In Section 2, we briefly survey prior work on multiple simultaneous collisions. In Section 3, we introduce some preliminary work on Gauss-Seidel solvers. In Section 4, we give some details of our generalized Gauss-Seidel solver. In Section 5, we demonstrate extensive experiments and comparisons to verify the robustness of our algorithm. In Section 6, we present the conclusion and future work.

2 RELATED WORK

Multi-body simulation and simultaneous collisions have gained significant attention in physically-based simulation and, therefore, have been extensively studied [Hahn 1988; Moore and Wilhelms 1988]. Here, we briefly review related work and refer readers to comprehensive surveys in [Andrews et al. 2022; Baraff 1993; Bender et al. 2014].

In rigid body simulation, three collision and impact strategies are commonly used, including constraint-based methods [Baraff 1989, 1994], penalty-based methods [Moore and Wilhelms 1988] and impulse-based methods [Mirtich and Canny 1995]. Constraint-based methods formulate the contacts and impacts using non-penetration constraints. Penalty methods typically introduce small abstract springs at contact points to prevent overlapping by applying spring forces. Impulse-based methods compute impulses instead of internal forces to satisfy non-penetration constraints. Among these methods, constraint-based methods are widely used in various physics engines due to their generality and the ability to handle complex scenarios. To handle simultaneous collisions, constraint-based methods either use synchronous time-stepping in which all violated constraints are considered as simultaneous collisions [Bridson et al. 2002], or asynchronous time instant in which the exact time of impact is computed for individual contacts and processed according to their temporal sequences [Lubachevsky 1991]. Considering practical applications, simultaneous strategies are often favored for handling multiple collisions within a single time step. The existing algorithms for handling multi-body collision can be further divided into two categories [Baraff 1989]: symmetry (i.e., addressing Bernoulli's problem) or propagation (i.e., addressing Newton Cradle problem).

Symmetry. In constraint-based methods, LCP solvers are particularly effective for handling contact and collision constraints, preserving simultaneity and symmetry by construction [Baraff 1989; Moreau 1985; Stewart 2000]. LCP solvers consider the initial relative velocities of all constraints simultaneously and ensure symmetric collisions under all restitution coefficients. In the elastic state, since the LCP solvers do not alter the initial relative velocities during iterations, they are unable to produce the phenomenon of impact propagation [Chatterjee and Ruina 1998; Glocker 2004]. If and only if the restitution coefficient is zero, the post-collision velocity obtained by the

LCP solution is correct [Smith et al. 2012]. In this case, the initial relative velocity does not affect the impulse calculation.

Propagation. The propagation model is particularly suitable for applying iterative methods. These methods sequentially resolve collisions independently of each other. The multiple simultaneous collisions are divided into a few simpler problems that involve a single collision between two bodies. After a contact is handled, subsequent objects will be handled using the new relative speed. During propagation, new violation constraints may occur one after another, and the same constraints may be solved multiple times. This process will continue until no violations are observed. By sacrificing physical accuracy, some solvers use a random order [Ivanov 1995] to handle multiple impacts. It is very likely to obtain a non-unique solution for the multi-object problem. The work in [Baraff 1989] may lead to an infinite set of collision calculations to complete one macroscopic simultaneous collision. Simultaneous collisions can also be ordered by normal approach velocity [Chatterjee and Ruina 1998]. Two linear iterative solvers, Jacobi and Gauss-Seidel, are well studied. Jacobi solver applies an impulse to each object after the impulses are solved for all contact constraints [Maclaurin 1742]. It solves each constraint independently, so it is naturally suitable for parallel solving. In addition, its solution meets the propagation and symmetry conditions under the convergence conditions. However, the Jacobi method will lead to an obvious non-conservation of kinetic energy. Gauss-Seidel solvers immediately apply the impulse to each object as soon as the impulses are computed. By incorporating some constraints, Gauss-Seidel solvers can be used to solve simultaneous problems, such as projected Gauss-Seidel solver in Bullet [Coumans and Bai 2021] or Gauss-Seidel-like solver proposed in [Erleben 2007]. These collision solvers can simulate contact breaking and impact propagation, yet ignore symmetry preservation. During propagation, these solvers may not be able to converge in a finite number of iterations. In the case of large mass ratios, the convergence rate of the propagation model is significantly slower than the simultaneous model [Baraff 1989]. In propagation, inelastic collapse may be observed [Baraff 1989; McNamara and Young 1994]. When the coefficient of restitution $c_r < 1$, as the scale increases, it may lead to non-convergence within a finite number of iterations. In addition, the propagation model may not terminate normally in the case of implicit equality [Vouga et al. 2017]. Therefore, additional treatments are required for the propagation model to ensure termination [Provot 1997].

To achieve both symmetry and propagation, some new hybrid methods were proposed [Chen et al. 2023; Smith et al. 2012]. Generalized reflection [Smith et al. 2012] maintains and solves a set of active constraints to simulate propagation until the set is empty. Intuitively, a subset is created using the current spatial configuration and relative velocity of the active constraints. Then, the algorithm solves the LCP locally using the projected Gauss-Seidel method for the given subset.

In this paper, we present a new robust and generalized Gauss-Seidel solver to overcome the difficulties in simultaneous collisions. We leverage spatial and temporal collision states (or collision trends) to classify and solve constraints, guaranteeing physically correct symmetry and collision propagation.

3 BACKGROUND

In this section, we will briefly introduce the concept of constrained dynamics and give some preliminaries on Gauss-Seidel-like solvers which our algorithm relies on.

3.1 Constrained Dynamics

A constraint is a function that imposes restrictions on the motion of an object. The equation that satisfies the constraint condition is called the constraint equation. In physics, constraints follow the laws of physics, including position constraint equations, velocity constraint equations, etc. Here,

we only introduce velocity constraint equations used by our solver. Refer to [Andrews et al. 2022] for more details.

In collision problems, a velocity constraint equation is usually formulated in the form of $C(V) = 0, C(V) \in \mathbb{R}^m, V \in \mathbb{R}^{6n}$, where $V = [v_1^\top \ \omega_1^\top \ v_2^\top \ \omega_2^\top \ \dots \ v_n^\top \ \omega_n^\top]^\top$ is the general representation of the velocity of all objects in the system, n is the number of objects in the system and m is the number of collision constraints. If we extract V from the equation, we get the form $C(V) = JV$, where $J = [J_1^\top \ J_2^\top \ \dots \ J_m^\top]^\top$ is called the Jacobian matrix, which represents the mapping relationship from the object space to the constraint space.

3.2 Linear Complementarity Problem

Linear Complementarity Problem describes a system where a set of linear equations satisfy the form $Ax + b \perp x$, and in constrained dynamics, constraint impulses and post-collision velocities exactly satisfy the relationship described by the following formula:

$$JV' = JM^{-1}J^\top \lambda + J\bar{V}_0 \quad (1)$$

$$JV' \geq 0, \quad \lambda \geq 0, \quad JV' \cdot \lambda = 0 \quad (2)$$

$$\bar{V}_0 = (1 + c_r) (V_0 + M^{-1}f_{\text{ext}} \cdot \Delta t) \quad (3)$$

Here, λ is the impulse to be applied to objects and JV' is the relative velocities along the contact normal. V_0 is the initial velocity at the beginning of each simulation step, and $J\bar{V}_0$ is the updated relative velocity affected by gravity, external forces f_{ext} and restitution coefficient c_r . M is a diagonal matrix composed of the mass matrix and the rotational inertia of each object in the system. Under constraints, feasible λ and JV' form an LCP satisfying several conditions [Baraff 1989]. (a) Impulses need to push the objects along the constraint normals instead of pulling them. (b) Normal velocities after the impulses are conducted need to stop objects from penetrating deeper. (c) Impulses occur only at contact points. If two objects are separating, there is no impulse between them. These conditions describe the complementarity of the two unknowns λ and JV' which correspond to the above equations.

If we substitute $JV' = 0$ into equation (1), we obtain a system of linear equations,

$$JM^{-1}J^\top \lambda = -J\bar{V}_0 \quad (4)$$

which represents the impulse required to correct the relative velocity to a separating state.

3.3 Projected Gauss-Seidel Solver

For the LCP, Gauss-Seidel-like solvers are widely used due to their efficiency in iteration. Note that in some literature, they are known as the iterative splitting method [Cottle et al. 2009; Erleben 2007; Vouga et al. 2017]. Among all the existing physics engines like Bullet [Coumans and Bai 2021], the project Gauss-Seidel (PGS) solver is used to bind unknowns while satisfying the LCP conditions.

Specifically, for a linear equation $Ax = b$, if A is split into $A = D - L - U$, where D is a diagonal matrix, L and U are the upper triangular matrix and lower triangular matrix respectively, then $Ax = b$ is equivalent to

$$Dx = (L + U)x + b \quad (5)$$

The iterative form of this equation can be written as

$$x^{k+1} = D^{-1}(L + U)x^k + D^{-1}b \quad (6)$$

, which is the Jacobi method. Writing this in a component form, we have

$$x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{i \neq j} a_{ij}x_j^{(k)}) \quad (7)$$

For Gauss-Seidel method, it will further use the previously obtained intermediate solution to calculate the new solution

$$x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)}) \quad (8)$$

For the LCP, since equation (2) restricts the range of λ , each iteration must project the solution onto the feasible region. Therefore, PGS is required, and the left-hand side of Equation 7 should be replaced with

$$x_i^{(k+1)} = \max(0, \frac{1}{a_{ii}}(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)})) \quad (9)$$

For equation (4), $A = JM^{-1}J^T$, $b = J\tilde{V}_0$ and $x = \lambda$, so PGS can be used to solve it. Unlike the Jacobi method, PGS iterations utilize results from the current iteration, meaning the components are not independent of each other.

Since the LCP treats collisions as occurring simultaneously, $J\tilde{V}_0$ cannot be changed during the process of solving the constraint equations for a time step, which causes the initial contact object to still exhibit the adhesion phenomenon after the collision [Smith et al. 2012].

4 GENERALIZED GAUSS SEIDEL SOLVER

In this section, we illustrate our generalized Gauss-Seidel solver for handling symmetry and propagation simultaneously. We first introduce the rules in our algorithm for classifying collision constraints using a state machine. We then elucidate the internal mechanism through a detailed case study.

4.1 State Machine

We divide constraints into four types in terms of the initial impulse and the current impulse for a contact. During each iteration, these constraints are updated in distinct ways, aiming to maintain symmetry in the initial setup while guaranteeing correct collision propagation.

As shown in Equation (8), in the PGS iteration, solving the impulse requires focusing on two known quantities: the initial relative velocity $J\tilde{V}_0$ (corresponding to the bias term b) and the current relative velocity $JV^{(k)}$ (which includes the application of the already computed iterative solution). During iteration, any constraint satisfies either its equation or not. Intuitively, this corresponds to the existence of an impulse at contact or not, based on these two quantities, we can classify the constraints into the following four states.

$$\begin{aligned} \text{STATE 1} \quad \text{Collision :} \quad & J\tilde{V}_0 < 0, \quad JV^{(k)} < 0 \\ \text{STATE 2 Pre-Separation :} \quad & J\tilde{V}_0 < 0, \quad JV^{(k)} \geq 0 \\ \text{STATE 3} \quad \text{Separation :} \quad & J\tilde{V}_0 \geq 0, \quad JV^{(k)} \geq 0 \\ \text{STATE 4} \quad \text{Pre-Collision :} \quad & J\tilde{V}_0 \geq 0, \quad JV^{(k)} < 0 \end{aligned}$$

For STATE 1, the initial impact and impulse are not completely resolved ($JV^{(k)} < 0$) at the k -th iteration. For STATE 2, the initial impact and impulse ($J\tilde{V}_0 < 0$) are resolved ($JV^{(k)} \geq 0$) in the k -th iteration. For STATE 3, there is no impulse initially and there is no impulse introduced during iteration. For STATE 4, there is no impulse initially, but a new impulse is introduced at the k -th iteration.

When we view the above inequalities from a physical perspective, we realize that these constraint states not only correspond to different states in the constraint-solving process but also represent

different stages of the collision process between a pair of objects: collision (STATE 1), pre-separation (STATE 2), separation (STATE 3), and pre-collision (STATE 4). STATE 1 signifies active collision and contact, STATE 2 represents the transition from contact to separation, and STATE 3 denotes the separated state. To account for collision propagation, where one collision triggers another, we introduce STATE 4, which occurs immediately before a propagated collision and exclusively precedes STATE 1. Non-propagated collisions do not influence pre-collision constraint solving and thus do not require a separate state.

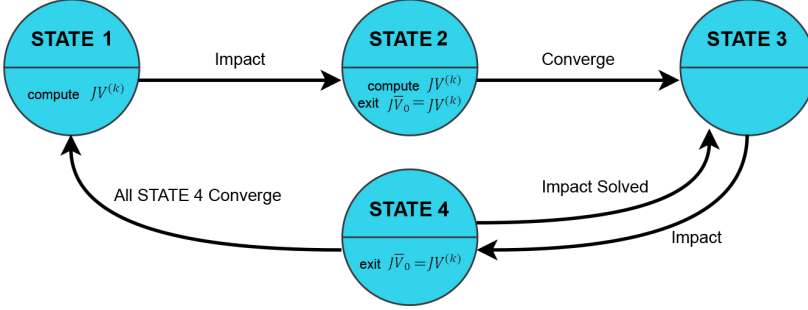


Fig. 2. The state machine and state transitions that occur as constraints are iteratively resolved. Any constraint belongs to one of these four states at a given time instant, and its state is determined by the impulse of a corresponding contact. When a constraint is satisfied, its state may transition from one to another. If all constraints are satisfied, the simulation converges for the given iteration, and the algorithm terminates.

Fig. 2 shows the state machine representing the transitions that occur as constraints are iteratively resolved. For a single constraint, during the iteration process, it will transition between one or more states. Globally, as the constraint states change, the collision propagates between objects. During this propagation process, our algorithm must ensure that symmetry is preserved, which involves maintaining synchronization between constraints.

Algorithm 1 lists the pseudocode of our generalized Gauss-Seidel solver, which incorporates the state transition in a standard Gauss-Seidel solver. This improvement allows our algorithm to perfectly handle simultaneous collisions that exhibit both symmetry and propagation.

In Algorithm 1, the transition from STATE 1 to STATE 2 occurs naturally during iteration, while the transition from STATE 2 to STATE 3 is explicitly performed by updating the initial velocity. As for the transition from STATE 3 to STATE 4, it is only triggered by collision propagation, just as a stationary object moves only when subjected to an external force. And the transition from STATE 4 to STATE 1 is explicitly performed, involving the synchronization of constraints, which is key to ensuring symmetry in collision propagation. We will further elaborate on this in Section 4.3.

The transition from STATE 4 to STATE 1 involves synchronization between constraints. According to the state transition, we can efficiently construct the local LCP and impact propagation. Unlike generalized reflection [Smith et al. 2012], we do not need additional procedures to construct the local LCP subset. Instead, the subset is automatically updated during the PGS solving process. Moreover, the iteration will terminate naturally if the solution meets the convergence criteria. This avoids the extra work of checking if any subset of constraints is still violated.

This state machine provides a crucial advantage by tracking intermediate stages of collision resolution. If the solution process is interrupted, we can seamlessly resume calculations based on the current state and movement of the objects. Our local solving method capitalizes on this state machine by ensuring that even if a global solution is not achieved, the propagation and symmetry properties of the local solution remain intact. For further details, refer to Section 4.4.

Algorithm 1 gGSsolver($\mathbb{V}^0, \mathbb{V}, \mathbb{C}, c_r$)

$\mathbb{V}^0$ [input]: initial velocity of all objects
 $\mathbb{V}$ [input]: velocity of all objects, affected by impulse
 $\mathbb{C}$ [input]: a set of contact constraints
 c_r [input]: a given coefficient of restitution
 $\mathbb{V}$ [output]: velocity of all objects after solution

```

1:  $k \leftarrow 0$ 
2: violate = true
3: while violate = true do
4:   violate = false
5:    $k \leftarrow k + 1$ 
6:   for  $\mathbb{C}_i$  in  $\mathbb{C}$  do
7:      $Jv_i^0 = J\mathbb{V}^0[i]$ 
8:      $Jv_i^k = \text{velocity}(\mathbb{V}, \mathbb{C}_i)$ 
9:     switch (state( $Jv_i^0, Jv_i^k$ ))
10:    case STATE 1:
11:       $\lambda = \lambda + \text{PGS}(\mathbb{V}, \mathbb{C}_i, c_r)$ 
12:       $\mathbb{V} = \text{update}(\lambda)$ 
13:      violate = true
14:    case STATE 2:
15:       $\lambda = \lambda + \text{PGS}(\mathbb{V}, \mathbb{C}_i, c_r)$ 
16:       $\mathbb{V} = \text{update}(\lambda)$ 
17:      if converge( $Jv_i^k$ ) then
18:        set  $Jv_i^0 = Jv_i^k$ 
19:        violate = true
20:      end if
21:    case STATE 3:
22:      continue
23:    case STATE 4:
24:      if converge( $Jv_i^k$ ) for all contacts then
25:        set  $Jv_i^0 = Jv_i^k$ 
26:        violate = true
27:      end if
28:    end switch
29:   end for
30: end while
31: return velocity of all objects.

```

4.2 Case Study: Symmetry and Propagation

In this section, we describe the mechanism in our algorithm using a simple scenario that exhibits both symmetry and propagation. As shown in Fig. 3, five objects are denoted by circles and symmetrically positioned. Two red objects (A and B) move towards three stationary objects C, D and E, causing four simultaneous collisions. Their velocities are denoted by arrows, with the length emphasizing the speed and direction of each object. We consider the configuration in Fig. 3-(a) as the moment of the first collision when collisions occur between A and C, and B and C. Here, we denote these two constraints as AC and BC. Constraints AC and BC are initially unsolved and carry

over to the successive iteration step. Therefore, these two constraints are in STATE 1. We compute their impulses using the Gauss-Seidel solver, in which a local LCP among objects A, B, and C is formulated. The order of handling constraints is relevant to the way we formulate them. Even though it is unknown in advance if constraints CD and CE are solved before or after AC and BC, it does not affect our solution. Consider the case in which constraints CD and CE are first solved in the first iteration. Since their relative velocity is zero and there is no initial impulse, constraints CD and CE are in STATE 3. The algorithm does not apply any impulse to change their velocity and their state. In the second iteration, due to the impact of constraints AC and BC, separating impulses are applied to C. This results in the violation of constraints CD and CE, transitioning them to STATE 4.

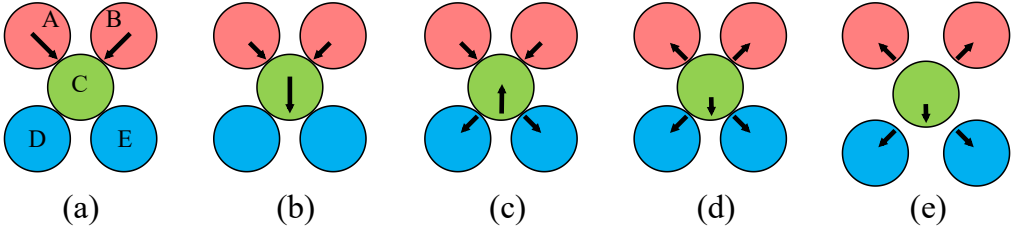


Fig. 3. The state transition process for simultaneous collision for five objects. (a) Two red objects (A and B) move towards three stationary objects. In the first impact, four collision constraints exist: AC, BC, CD and CE. Constraints AC and BC are in STATE 1. CD and CE are in STATE 3. (b) After iterations, AC and BC transition from STATE 1 to STATE 2, and then to state 3. CD and CE transition from STATE 3 to STATE 4, and then to STATE 1. (c) CD and CE transition from STATE 1 to STATE 2, and then to STATE 3. AC and BC transition from STATE 3 to STATE 4, then to STATE 1. (d) The final state after solving all collisions. AC and BC transition from STATE 1 to STATE 2, and then to STATE 3. CD and CE are already in STATE 3. (e) Since there are no violation constraints, our algorithm terminates and the simulation proceeds symmetrically.

In STATE 4, objects are not directly affected by impulses because STATE 4 is a transitional state between STATE 3 and STATE 1. When object C is influenced by constraints AC and BC, its velocity does not change immediately. Intuitively, the velocity of object C does not achieve the magnitude shown in Fig. 3-(b) after a single-step iteration only. Therefore, the constraints in STATE 4 cannot apply impulses until the velocity of object C converges and the other objects in STATE 4 converge as well. Both conditions are necessary for synchronization, as any violation may destroy symmetry in successive simulations. However, the second condition in this example is natural once the relative velocity converges. The convergence in relative velocity implies that the change between the previous and current iterations is negligible. When both conditions are satisfied, a constraint transitions from STATE 4 to STATE 1, and the initial relative velocity is updated using the current relative velocity. Constraints CD and CE are constructed under the condition of having the same initial relative velocity and considering impulse propagation. The second condition of STATE 4 is related to synchronization requirements in more complex scenarios.

In Fig. 3, the transition from (b) to (c) is similar to that from (a) to (b). Constraints CD and CE are in STATE 1 and involved in local LCP, while AC and BC transition from STATE 3 to STATE 4. In Fig. 3, from (c) to (d), the constraints AC and BC transition from STATE 1 to STATE 2 and then to STATE 3. Once they transition to STATE 3, there are no violation constraints and our algorithm terminates. Fig. 3-(e) shows that the simulation continues without collision. This case study covers the scenarios of symmetry and propagation and involves transitions between four states, demonstrating the mechanism of our algorithm. Although efficient and robust, our implementation is very simple.

Algorithm 1 demonstrates the pseudo-code incorporating four states and the transition between them in a standard Gauss-Seidel solver.

4.3 Further Discussion on STATE 4

To transition from STATE 4 to STATE 1, all other objects in STATE 4 must converge. Here, we give another example to illustrate this transition.

As shown in Fig. 4, eight objects (A~H) are symmetrically placed. When two red objects (A and B) move towards the other six stationary objects. At the beginning, the handling is exactly the same as in Fig. 3. Constraints CD and CE transition to STATE 1 when the green ball converges. Solving these two constraints results in the green ball gradually moving in the opposite direction, as shown in Fig. 4-(c).

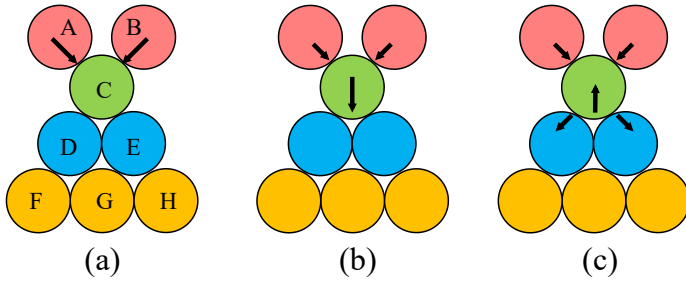


Fig. 4. Transition from STATE 4 to STATE 1. (a) Two red objects (A and B) move towards the other six stationary objects. The handling is exactly the same as in 3-(a). (b) Green balls and blue balls are in STATE 1 and will transition to the state shown in (c). (c) Constraints AC and BC are in STATE 4, as are constraints DF, DG, EG, and EH. There are six constraints in total, and our algorithm permits the transition from STATE 4 to STATE 1 only if all constraints converge.

Meanwhile, new violations occur between the blue and orange balls. As a result, new constraints DF, DG, EG, and EH are generated in STATE 4. Constraints AC and BC are also generated in STATE 4. In total, there are six constraints in STATE 4. Observe that some constraints may converge earlier than others. If we transition the early converged constraints from STATE 4 to STATE 1 when some other constraints are still in STATE 4, this may generate unexpected collisions and destroy the simultaneity and symmetry in subsequent simulations. This observation tells that even if some constraints in STATE 4 reach its convergence condition, we still need to wait until all other constraints in STATE 4 converge before transitioning to STATE 1.

An exception may occur when two balls are in the same row. They may have a positive relative velocity during iteration. However, this intermediate velocity can be quickly corrected. Consequently, we may observe that some constraints transition from STATE 3 to STATE 4 and then quickly transition back to STATE 3. In terms of implementation, in each iteration, we carefully check the number of constraints in STATE 4.

4.4 Insufficient iterations and local propagation

Our state machine terminates when all constraints are satisfied. However, this ideal scenario may assume sufficient iterations. In practice, solving all constraints becomes increasingly challenging as the scene's scale and geometric complexity grow, necessitating a larger number of iterations. Consequently, the constraint solver may need to terminate prematurely. This section addresses how to ensure that the solution maintains symmetry and propagation properties under insufficient iterations.

By treating propagation as complete between substeps, we can restructure our algorithm to use substeps as the fundamental time intervals. Our approach differs from substepping in XPBD [Macklin et al. 2019] which replaces iterations and essentially calls the PGS solver multiple times with shorter durations. In contrast, our algorithm still involves multiple iterations within each substep to guarantee the symmetry and propagation properties of the local solution.

Since time steps only propagate interactions between adjacent objects, we define substep completion when all current constraints are either in a separated state or in a pre-collision state and have reached convergence. This criterion ensures the symmetry properties of the local solution, as the pre-collision convergence conditions inherently encompass synchronization and symmetry requirements.

While substep completion criteria are well-defined, computational limitations may impose an upper limit on the number of iterations per time step. If this limit is reached before all constraints converge, the solver is likely still processing a particular substep ($i + 1$). In this case, the solution is rolled back to the state of the previous substep (i), which represents a spatially symmetric solution after i rounds of propagation. The intermediate state of the unfinished substep $i + 1$ is then saved for the next time step, providing a *warm start* for its initial substep. In other words, upon substep convergence, the solution is updated with the substep result. If a substep does not converge within the time step limit, the state is rolled back to the previous substep's solution, and the unfinished substep is continued in the next time step. This approach ensures the robustness of the solution result.

By manually adjusting the substep size, we effectively control the extent of propagation within each time step. This feature allows users to tailor the propagation behavior of objects according to specific simulation requirements.

5 EXPERIMENTAL RESULTS & COMPARISON

In this section, we give some details of our implementation and highlight the performance against a standard Gauss-Seidel, Jacobi and generalized reflection (GR) [Smith et al. 2012].

5.1 Implementation

Our algorithm was implemented in C++ and tested on an Intel i5-13600KF CPU (14 cores, 3.50GHz) with 32GB memory. For comparison, we use the Gauss-Seidel in Bullet [Coumans and Bai 2021; Ferguson et al. 2021], and generalized reflection (GR) [Smith et al. 2012].

5.2 Ball Geyser

In this experiment, a set of balls is symmetrically placed between two stationary obstacles. A red ball is moving upward along the axis of symmetry, striking other balls. Our algorithm can guarantee the symmetry property during impact propagation in the case of simultaneous and symmetric multiple collisions. Please refer to Fig. 1 and watch the accompanying video for the entire simulation.

5.3 Billiard Balls

Our algorithm was tested in a scenario in which billiard balls were arranged in 10 layers to verify symmetry and propagation. In our experiment, the cue ball strikes the pile of billiard balls along the symmetry axis with a given initial velocity. All balls have an identical radius of 1. As shown in Fig. 5, the symmetry is well preserved. In our experiment, the balls originally in a symmetrical arrangement remain symmetric after collision, with an error tolerance of less than 0.01, even over thousands of simulation steps.

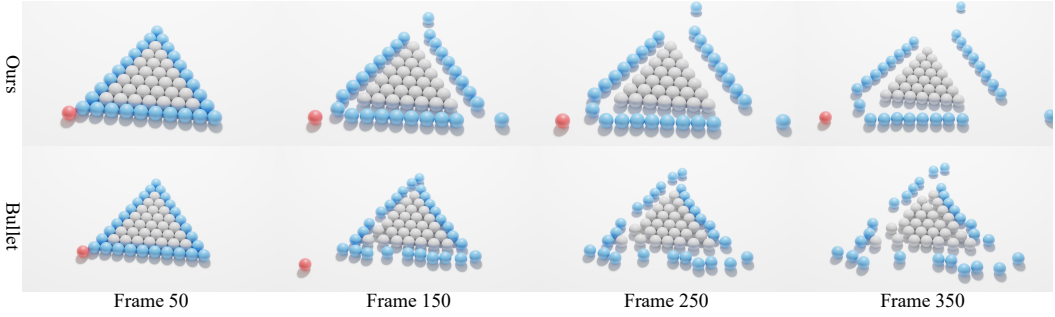


Fig. 5. Performance comparison with Bullet. A red ball, with an initial velocity of 5 m/s, moves from left to right. At frame 50, the cue ball impacts the first ball. Our algorithm maintains symmetry over hundreds of steps. Note: Our algorithm and the GR solver yield identical results.

Our algorithm also guarantees correct impact propagation. When the cue ball strikes the first ball in the pile, the balls in the middle remain stationary and only the balls on the outermost move apart, which is physically correct[Smith et al. 2012]. Watch the accompanying video for the simulation.

We also show the performance comparison of our algorithm with that of Bullet [Coumans and Bai 2021]. As shown in Fig. 5, we conducted the same experiment in Bullet and used the same tolerance and number of iterations (e.g., 40). In Bullet, the symmetry is no longer guaranteed merely after a few simulation steps and its collision propagation is incorrectly conducted.

5.4 Wall Rebound

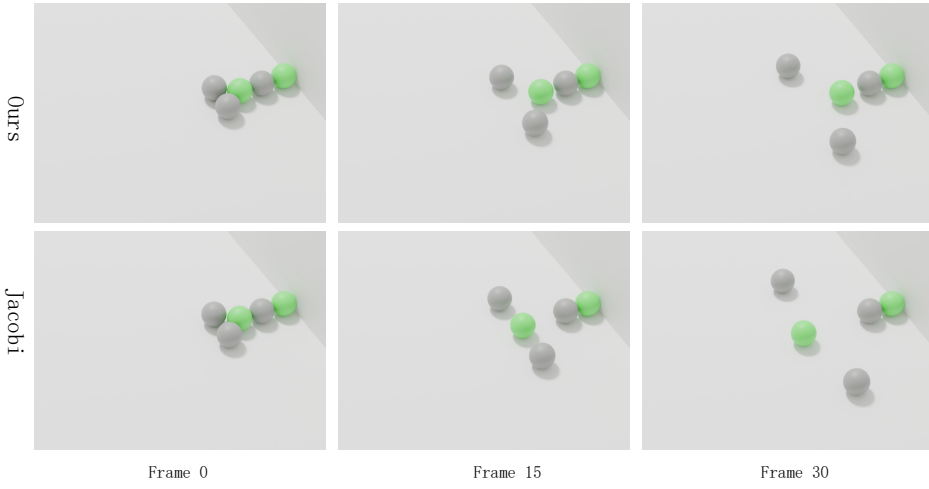
In Fig. 6(a), two gray balls collide simultaneously with a green ball, triggering a rebound chain reaction. We compared our algorithm's performance with the Jacobi solver under identical initial conditions. And Fig. 6(b) demonstrates that the Jacobi solver produces significantly higher total kinetic energy in post-collision compared to our algorithm. This observation highlights a key limitation of the Jacobi solver, namely its inability to consistently maintain kinetic energy conservation.

5.5 Local Symmetry and Impact Propagation

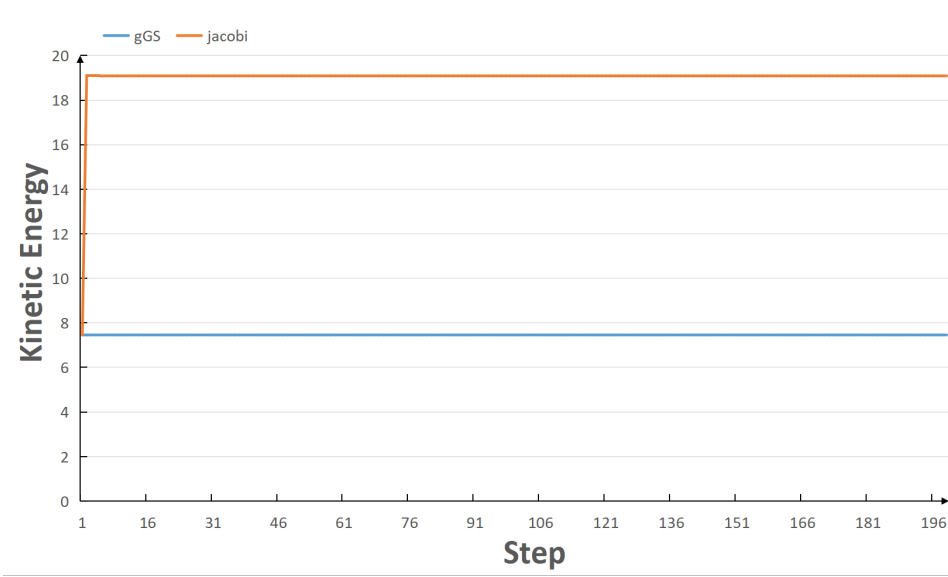
To address computational limitations, our algorithm incorporates a substep method that concurrently provides a mechanism for controlling propagation. This approach aligns with the observation that real-world object interactions exhibit limited propagation capabilities [Hascoët et al. 1999; Herrmann and Seitz 1982]. We present two sets of experiments to demonstrate the effectiveness of our local propagation function.

As depicted in Fig. 7, we evaluated our partial and controllable propagation mechanism within a Newton cradle scenario comprising 17 balls. By setting the maximum propagation limit to 3, the iterative process ends with the impact propagating to the third ball. The collisions involving the remaining balls are then processed in subsequent time steps. In these subsequent steps, gaps may emerge between the balls due to the position and velocity updates and the absence of explicit position correction. Despite this, our algorithm still effectively simulates the cascading effect of impact propagation in multiple time steps.

Furthermore, Fig. 8 presents a comparative experiment with GR, where a red ball impacts a symmetric diamond-shaped array of balls with an initial velocity of 5 m/s. Here, we restricted the propagation upper limit to 3. However, the results demonstrate negligible differences between our



(a) Wall Rebound



(b) Kinetic Energy

Fig. 6. Comparison between our algorithm and the Jacobi solver. (a) A scenario where two gray balls simultaneously strike a green ball. (b) Post-collision kinetic energy comparison, showing that the Jacobi solver exhibits significantly higher kinetic energy than our solver, indicating potential energy conservation issues.

algorithm and GR. This observation empirically validates that our algorithm effectively preserves the desirable properties of the solution even when employing local propagation.

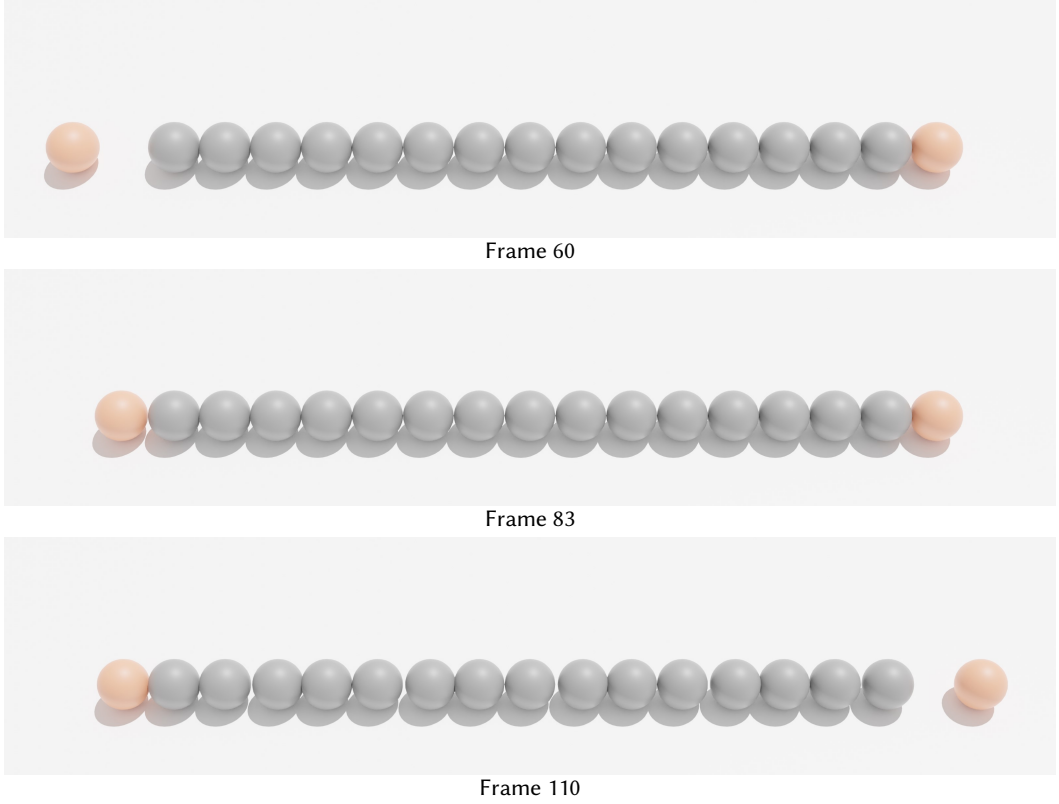


Fig. 7. Newton's cradle with propagation upper limit to 3. The propagation can still be conducted over multiple steps, but only up to three objects are propagated at a time and there are spatial gaps near the balls where the propagation ended in the previous step.

5.6 Numerical Errors

Inelastic collapse is inevitable in practical scenarios. When the coefficient of restitution (c_r) is less than 1, each propagation reduces the relative speed. When the speed drops below a certain threshold, the impulse cannot be applied effectively due to floating-point truncation errors. Consequently, constraints remain in a violated state regardless of velocity updates, causing the iteration to fail to converge within a finite number of steps. Even in elastic collision scenarios, an excess of oblique collisions can accumulate numerical errors, resulting in small, persistent values that degrade convergence. These small values behave like "ghosts", oscillating within the constraints and significantly impacting convergence performance. Without proper handling, increasing the scale of the scene exacerbates the problem, preventing iteration convergence.

These problems are primarily caused by floating-point precision errors. We can detect the collapse phenomenon by calculating the trend of velocity changes. In our algorithm, such collapse can only occur in STATE 1, where objects are in a violated state and impulses are generated. Therefore, specific treatments are needed for the collapse phenomenon in STATE 1.

One approach is to directly round off the velocities. Since the speed decreases during the transition, rounding off small velocities that cause truncation errors can be considered to achieve convergence. While simple and effective, this method leverages floating-point precision defects. Assuming infinite

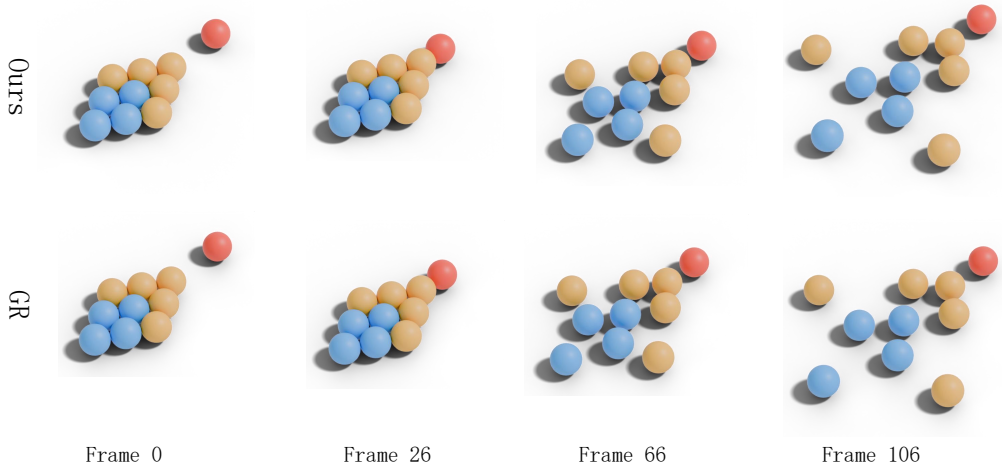


Fig. 8. Performance comparison with GR. Top: our algorithm. Bottom: GR. A red ball hits the symmetrical diamond ball array with an initial velocity of 5 m/s. Our algorithm sets the propagation upper limit to 3, but still achieves the same effect as GR.

floating-point precision, such a threshold would be unattainable, preventing iteration termination. Alternatively, we can adopt the interpolation method used in the GR solver. For $c_r = 0$, we use the global LCP to solve the constraints, and for $c_r > 0$, we calculate the speed through interpolation. This guarantees convergence for any restitution coefficient.

As discussed in Section 4, convergence in STATE 2 and STATE 4 is determined by the trend of changes in relative speed. When the trend is below a given tolerance, the constraint is considered to be convergent. A non-zero tolerance allows for earlier convergence but sacrificing precision. This may be useful if users require visually corrected effects with a low accuracy. Our algorithm effectively manages inelastic collapse and improves convergence performance, maintaining both accuracy and efficiency.

To verify numerical stability, tracking the variation of the kinetic energy during simulation is a common evaluation method. In the supplementary material, we provide line charts of kinetic energy variation for PGS, Jacobi, and our algorithm under the same scenario to demonstrate the numerical stability of our approach.

5.7 More Results

Due to space limitations, we have included additional work in the supplementary material. The readers can find more experimental results there, including collisions with rotation, collisions involving various shapes, and comparison experiments with MuJoCo.

6 CONCLUSION

We present a simple yet robust Gauss-Seidel solver capable of effectively handling simultaneous collisions and responses that exhibit symmetry and propagation. Owing to its simplicity, our approach can be seamlessly incorporated into existing physics engines such as Bullet, ODE and Box2D, enabling promising simulations involving simultaneous collisions. Furthermore, by employing a local propagation algorithm, our method maintains desirable dynamic properties even when global convergence is not attained, while also granting users control over the extent of propagation.

Limitations and Future Work. Our algorithm can partially detect the occurrence of numerical errors and mitigate their impact. However, our algorithm still faces uncertainties that arise from other sources of numerical problems that may introduce unnecessary iterations. In future work, we would like to extend our algorithm to address more complex scenarios characterized by intricate friction and rotational dynamics, which are generally considered highly challenging in physics simulations. For complex-shaped bodies, we anticipate that our algorithm will exhibit comparable performance, contingent upon the reliability of the underlying collision detection. Furthermore, for large-scale scenarios, performance may be enhanced by employing spatial data structures to efficiently store and retrieve the propagation history of collisions.

ACKNOWLEDGMENTS

This work was supported by the National Science and Technology Major Project (2021ZD0114501).

REFERENCES

- Sheldon Andrews, Kenny Erleben, and Zachary Ferguson. 2022. Contact and friction simulation for computer graphics. *ACM SIGGRAPH 2022 Courses*, Article 3 (2022).
- David Baraff. 1989. Analytical methods for dynamic simulation of non-penetrating rigid bodies. *ACM SIGGRAPH* 23, 3 (1989), 223–232.
- David Baraff. 1993. Non-penetrating rigid body simulation. *Eurographics* (1993).
- David Baraff. 1994. Fast contact force computation for nonpenetrating rigid bodies. *ACM SIGGRAPH* (1994), 23–34.
- Jan Bender, Kenny Erleben, and Jeff Trinkle. 2014. Interactive simulation of rigid body dynamics in computer graphics. *Computer Graphics Forum* 33, 1 (2014), 246–270.
- Robert Bridson, Ronald Fedkiw, and John Anderson. 2002. Robust treatment of collisions, contact and friction for cloth animation. *ACM Transactions on Graphics* (2002), 594–603.
- Anindya Chatterjee and Andy Ruina. 1998. A new algebraic rigid-body collision law based on impulse space considerations. *Journal of Applied Mechanics* 65, 4 (1998), 939–951.
- Yi-Lu Chen, Mickaël Ly, and Chris Wojtan. 2023. Unified treatment of contact, friction and shock-propagation in rigid body animation. In *ACM Symposium on Computer Animation*. Article 5, 2 pages.
- Richard W Cottle, Jong-Shi Pang, and Richard E Stone. 2009. *The linear complementarity problem*. SIAM.
- Erwin Coumans and Yunfei Bai. 2016–2021. PyBullet, a Python module for physics simulation for games, robotics and machine learning.
- Kenny Erleben. 2007. Velocity-based shock propagation for multibody dynamics animation. *ACM Transactions on Graphics* 26, 2 (2007), 12–es.
- Zachary Ferguson, Minchen Li, Teseo Schneider, Francisca Gil-Ureta, Timothy Langlois, Chenfanfu Jiang, Denis Zorin, Danny M. Kaufman, and Daniele Panozzo. 2021. Intersection-free Rigid Body Dynamics. *ACM Transactions on Graphics* 40, 4 (2021), 183:1–183:16.
- Ch Glocker. 2004. Concepts for modeling impacts without friction. *Acta Mechanica* 168 (2004), 1–19.
- Eran Guendelman, Robert Bridson, and Ronald Fedkiw. 2003. Nonconvex rigid bodies with stacking. *ACM Transactions on Graphics* 22, 3 (2003), 871–878.
- James K Hahn. 1988. Realistic animation of rigid bodies. *ACM SIGGRAPH* 22, 4 (1988), 299–308.
- David Harmon, Etienne Vouga, Breannan Smith, Rasmus Tamstorf, and Eitan Grinspun. 2009. Asynchronous contact mechanics. In *ACM SIGGRAPH 2009 Papers*. Article 87, 12 pages.
- Erwan Hascoët, Hans J. Herrmann, and Vittorio Loreto. 1999. Shock propagation in a granular chain. *Physical Review E* 59, 3 (1999), 3202–3206.
- F. Herrmann and M. Seitz. 1982. How does the ball-chain work? *American Journal of Physics* 50, 11 (1982), 977–981.
- AP Ivanov. 1995. On multiple impact. *Journal of Applied Mathematics and Mechanics* 59, 6 (1995), 887–902.
- Boris D Lubachevsky. 1991. How to simulate billiards and similar systems. *J. Comput. Phys.* 94, 2 (1991), 255–283.
- Miles Macklin, Kier Storey, Michelle Lu, Pierre Terdiman, Nuttapong Chentanez, Stefan Jeschke, and Matthias Müller. 2019. Small steps in physics simulation. In *Proceedings of the 18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. Article 2, 7 pages.
- Colin Maclaurin. 1742. *A treatise of fluxions: in two books*. Vol. 1. Ruddimans.
- Sean McNamara and William R. Young. 1994. Inelastic collapse in two dimensions. *Physical review. E* 50, 1 (1994), R28–R31.
- Brian Mirtich and John Canny. 1995. Impulse-based simulation of rigid bodies. In *Symposium on Interactive 3D Graphics*. 181–ff.

- Matthew Moore and Jane Wilhelms. 1988. Collision Detection and Response for Computer Animation. *ACM SIGGRAPH* 22, 4 (1988), 289–298.
- J. J. Moreau. 1985. Standard Inelastic Shocks and the Dynamics of Unilateral Constraints. In *Unilateral Problems in Structural Analysis*, Gianpietro Del Piero and Franco Maceri (Eds.). Springer, 173–221.
- Ngoc Son Nguyen and Bernard Brogliato. 2014. *Multiple Impacts in Granular Chains*. Springer Berlin Heidelberg, 11–29.
- Xavier Provot. 1997. Collision and self-collision handling in cloth model dedicated to design garments. In *Computer Animation and Simulation '97*. Springer Vienna, 177–189.
- Stephane Redon, Ming C. Lin, Dinesh Manocha, and Young J. Kim. 2005. Fast Continuous Collision Detection for Articulated Models. *Journal of Computing and Information Science in Engineering* 5, 2 (2005), 126–137.
- Breannan Smith, Danny M Kaufman, Etienne Vouga, Rasmus Tamstorf, and Eitan Grinspun. 2012. Reflections on simultaneous impact. *ACM Transactions on Graphics* 31, 4 (2012), 106:1–106:12.
- David E Stewart. 2000. Rigid-body dynamics with friction and impact. *SIAM review* 42, 1 (2000), 3–39.
- Etienne Vouga, Breannan Smith, Danny M Kaufman, Rasmus Tamstorf, and Eitan Grinspun. 2017. All’s well that ends well: guaranteed resolution of simultaneous rigid body impact. *ACM Transactions on Graphics* 36, 4 (2017), 151:1–151:19.
- Xinyu Zhang, Stephane Redon, Minkyung Lee, and Young J. Kim. 2007. Continuous Collision Detection for Articulated Models using Taylor Models and Temporal Culling. *ACM Transactions on Graphics* 26, 3 (2007), 15:1–15:10.
- Danyong Zhao, Yijing Li, and Jernej Barbič. 2016. Asynchronous implicit backward Euler integration. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. 1–9.